

CONFIDENTIAL — INTERNAL PRESENTATION

NexoraDB

System Architecture Document | معماری سیستم

Core / هسته

C++20

Status / وضعیت

In Development

Date / تاریخ

2025

Version / نسخه

MVP — 1.0.0

GB ENGLISH

NexoraDB is a **Hybrid Graph–Document Database** with three core traits

- Stores documents (*like MongoDB*)
- Manages relationships as a graph (*like Neo4j*)
- C++ core for speed, Python interface for ease-of-use

IR فارسی

NexoraDB یک پایگاه داده هیبرید گرافی-اسنادی است که سه ویژگی اصلی دارد:

- اسناد را ذخیره می‌کند (*مانند MongoDB*)
- روابط را به صورت گراف مدیریت می‌کند (*مانند Neo4j*)
- هسته ++C برای سرعت، رابط Python برای راحتی

● کاربرد اصلی / Primary Use Case

شبکه اجتماعی و پیام‌رسان با میلیون‌ها کاربر، نیاز به کوئری‌های بلادرنگ روی گراف اجتماعی
Social network & messaging platform with millions of users requiring real-time social graph queries

● مزیت اصلی / Key Advantage

یک سیستم واحد به جای ترکیب MongoDB + Neo4j + Redis — کاهش پیچیدگی و latency
Single system instead of MongoDB + Neo4j + Redis combo — reduced complexity and latency



⚠️ قانون طلایی معماری / Golden Architecture Rule

IR RocksDB تنها منبع حقیقت است. LiveGraph فقط یک projection در RAM است که قابل rebuild از روی داده‌های دیسک است.

GB RocksDB is the sole source of truth. LiveGraph is merely an in-memory projection — fully rebuildable from disk.



NexoraDB/

— CMakeLists.txt	← Build orchestrator
— vcpkg.json	← Dependencies: rocksdb, fmt
— core/	← DocEngine (Storage Team)
— DocEngine.h	← Public API + DBResult + Schema/Index/FK defs
— DocEngine.cpp	← RocksDB impl, CRUD, WAL-backed transactions
— query/	← QueryLayer (Parser + Storage Team)
— Condition.h	← Condition tree (Leaf / Composite)
— UpdateSpec.h	← UpdateSpec (\$set, \$inc, \$push...)
— Evaluator.h	← IBsonAdapter, Evaluator interface
— Evaluator.cpp	← JsonAdapter + full Match/Apply impl
— graph/	← GraphEngine (Parser + Algorithm Team)
— GraphTypes.h	← All shared types: DenseId, NodeRecord, WalRec
— GraphStorage.h/cpp	← Disk layer: .nex / .nexr / .nexl files
— GraphWAL.h/cpp	← Write-Ahead Log with retention policy
— LiveGraph.h/cpp	← In-memory adjacency list + DenseId + FreeStac
— algorithms/	← Algorithm Team fills this folder
— AlgorithmBase.h/cpp	← LockAlgorithm / JobAlgorithm / StaticGraphVie
— CommonFollowers.cpp	← Algorithm Team
— PageRank.cpp	← Algorithm Team
— bindings/	← Python Integration (FastAPI Team)
— nexoradb_bind.cpp	← pybind11 - C++ → Python
— setup.py	← pip install nexoradb
— api/	← FastAPI Backend (FastAPI Team)
— main.py	← FastAPI app entry point
— routes/	← Endpoint routers
— parser/	← NexoraQL parser (Parser Team)
— nexql_parser.py	← CREATE GRAPH, INSERT INTO, SELECT...
— dashboard/	← React Frontend (Dashboard Team)
— src/	

└─ package.json

1 راه اندازی DocEngine

Initialize DocEngine

DocEngine engine("/var/data/nexoradb"); — RocksDB
 باز می شود، فایل های DB آماده می شوند، اگر پوشه وجود نداشت ایجاد می شود.

2 بررسی سلامت

Health Check

engine.IsHealthy() — اگر false باشد سیستم متوقف می شود. در صورت خطای RocksDB،
 std::runtime_error پرتاب می شود.

3 بررسی و بارگذاری Meta فایل گراف

Check graph_meta.nexl

فایل graph_meta.nexl بررسی می شود. اگر وضعیت Dirty یا Corrupted بود، rebuild خودکار انجام می شود. اگر Clean بود، داده ها از فایل های nex/.nexr. بارگذاری می شوند.

4 ساخت LiveGraph در RAM

Build LiveGraph in RAM

NodeRecord — LiveGraph.loadFromDisk() ها و EdgeRecord ها اسکن می شوند.
 رکوردهای ACTIVE وارد RAM می شوند. رکوردهای DELETED وارد FreeStack می شوند.

5 Replay WAL

Replay Write-Ahead Log

entries — LiveGraph.replayWAL() با applied=false روی گراف اعمال می شوند.
 consistency بین RAM و دیسک تضمین می شود.

6 سیستم آماده دریافت درخواست

System Ready

هر دو موتور آماده اند: engine.InsertOne() ، engine.FindMany() ،
 graph.addEdge() ، graph.neighbors() و غیره.

GB DESCRIPTION

NexoraQL is an SQL-like language. The Parser Team parses it and calls the corresponding C++ function. No logic resides in the parser — it is purely a routing layer.

IR توضیح

NexoraQL یک زبان با سینتکس نزدیک به SQL است. تیم Parser آن را parse می‌کند و تابع ++C مربوطه را فراخوانی می‌کند. هیچ منطقی در Parser نیست — فقط routing.

```
-- Document Operations INSERT INTO users VALUES
('{"username":"ali","email":"ali@ex.com"}'); SELECT * FROM users WHERE age > 18
LIMIT 10; UPDATE users SET bio = 'Hello' WHERE _id = 'u_001'; DELETE FROM users
WHERE active = false; -- Schema & Index CREATE COLLECTION posts (title STRING
REQUIRED, authorId STRING REQUIRED); CREATE INDEX idx_author ON posts
(authorId); ADD FOREIGN KEY fk_author ON posts (authorId) REFERENCES users
(_id); -- Graph Operations CREATE LIVE GRAPH social_graph; CREATE STATIC GRAPH
analytics; GRAPH social_graph MAP NODE User FROM users KEY _id; GRAPH
social_graph MAP EDGE FOLLOWS FROM follows SOURCE fromUserId AS User TARGET
toUserId AS User; TRAVERSE User("u_001") OUT FOLLOWS DEPTH 2 LIMIT 50; --
Algorithm Jobs RUN JOB PageRank ON social_graph (iterations=20, damping=0.85);
RUN LOCK CommonFollowers ON social_graph ("u_001", "u_002"); -- Transactions
BEGIN TRANSACTION; INSERT INTO messages VALUES ('{"text":"hi"}'); COMMIT;
```

جریان پردازش کوئری / Query Processing Flow

NexoraQL String

- Parser (Python) — tokenize & route
- pybind11 — cross language bridge
- C++ function call (DocEngine / LiveGraph / Algorithm)
- DBResult / AlgoResult
- FastAPI response (JSON)
- React Dashboard display

Write Path / نوشتن ●Python: `insert("users", doc)`

```

pybind11 ↓
() DocEngine::InsertOne
Schema Validation ↓
FK Constraint Check ↓
RocksDB::Put (BSON) ↓
Index Update ↓
WAL entry (graph.wal) ↓
LiveGraph::addNode() [if LIVE] ↓
GraphStorage::writeNode() (.nex) ↓
() WAL::markApplied ↓
DBResult { success=true, data=doc_id } →

```

Read Path / خواندن ●Python: `find("users", {"age":{"$gt":18}})`

```

pybind11 ↓
() DocEngine::FindMany
Index Scan (if available) ↓
Full Scan (if not indexed) ↓
Evaluator::Match(bson, condition) ↓
() Evaluator::ApplyProjection ↓
DBResult { success=true, data=[...] } →

:Graph Read
LiveGraph::neighbors("u_001", OUT)
vector<DenseId> [0(degree)] →

```

تصمیم / Decision	دلیل / Reason	جایگزین رد شده / Rejected
RocksDB به عنوان storage	LSM-tree، فشرده سازی Transaction، Snappy، built-in	LevelDB (بدون Transaction)
pybind11 به جای Cython	ساده تر، type-safe، مستندات بهتر، C++20 compatible	Cython (پیچیده تر، نیاز به pxd جداگانه)
Dense-ID در LiveGraph	دسترسی O(1) به array، cache-friendly، RAM کمترین	string-keyed hash map (کُندتر)
Fixed-size records در nex/.nexr.	دسترسی O(1) با offset، بدون index جداگانه	فرمت variable-length (نیاز به index)
WAL جداگانه برای گراف	consistency مستقل از RocksDB WAL، retention policy سفارشی	اتکا به RocksDB WAL مستقیم
Chunked Sorted Vector	برای heavy nodes (degree > 1024)، جلوگیری از O(n) shift	vector ساده (کُند برای influencers)
FastAPI به جای Flask	async native، validation، Swagger UI، built-in	Flask (sync، بدون validation)